

Computer Vision

HW4: Structure from Motion (SfM)

Group 18

312552026 蔡濟謙, 313552049 鄭博涵, 110550035 陳奎元

1. Introduction

Structure from Motion (SfM) is a photogrammetric range imaging technique used to estimate three-dimensional structures from sequences of two-dimensional images. These images may also incorporate local motion signals to enhance the process. SfM is a significant area of study within the fields of computer vision and visual perception due to its applications in reconstructing 3D models from 2D data. In biological vision, SfM refers to the phenomenon by which humans and other organisms can infer three-dimensional structures from the projected two-dimensional motion fields of moving objects or scenes, demonstrating its importance in understanding depth and spatial awareness.

2. Implement Procedure

- **find out correspondence across images**

We get the keypoints of the two images by SIFT, and then match them by KNN with ratio distance filter.

```
gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

image1, image2, kp1, kp2, des1, des2 = keypoints(image1, image2, gray1, gray2, output_dir, name)
src_pts, dst_pts = matching(kp1, kp2, des1, des2, threshold=0.66)
draw_matching(image1, image2, src_pts, dst_pts, output_dir, name)
```

```
def keypoints(image1, image2, gray1, gray2, output_dir, name):
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(gray1, None)
    kp2, des2 = sift.detectAndCompute(gray2, None)

    image1 = cv2.drawKeypoints(gray1, kp1, image1)
    image2 = cv2.drawKeypoints(gray2, kp2, image2)

    for i, image in enumerate((image1, image2), start=1):
        plt.subplot(1, 2, i)
        plt.imshow(image)
        plt.title(f'Image {i}')
        plt.axis('off')
    plt.savefig(f"{output_dir}/{name}/{name}_keypoints")
    plt.close()

    return image1, image2, kp1, kp2, des1, des2
```

```
def matching(kp1, kp2, des1, des2, threshold=0.5):
    matches = cv2.BFMatcher().knnMatch(des1, des2, k=2)
    good = [m for m, n in matches if m.distance < threshold * n.distance]

    src_pts = np.asarray([kp1[m.queryIdx].pt for m in good])
    dst_pts = np.asarray([kp2[m.trainIdx].pt for m in good])

    return src_pts, dst_pts
```

```

def draw_matching(image1, image2, src_pts, dst_pts, output_dir, name):
    image = np.concatenate((image1, image2), axis=1)

    n, _ = src_pts.shape
    h1, w1, c1 = image1.shape

    for i in range(n):
        color = [np.random.randint(0, 255) for _ in range(3)]
        x1 = int(src_pts[i, 0])
        y1 = int(src_pts[i, 1])
        cv2.circle(image, (x1, y1), 7, color, 1)

        x2 = w1 + int(dst_pts[i, 0])
        y2 = int(dst_pts[i, 1])
        cv2.circle(image, (x2, y2), 7, color, 1)

        for x in range(x1, x2):
            y = y1 + ((x - x1) / (x2 - x1) * (y2 - y1))
            image[int(y), x] = color

    plt.imshow(image)
    plt.axis('off')
    plt.savefig(f"{output_dir}/{name}/{name}_matching")
    plt.close()

```

- **estimate the fundamental matrix across images (normalized 8 points)**

The normalized 8-point algorithm computes the fundamental matrix by solving a system of homogeneous linear equations $x^T F x' = 0$ using SVD, with the additional constraint $|F| = 0$.

To handle outliers, we incorporate RANSAC and the Sampson error into the process.

```
def fundamental_matrix(pts1, pts2, maxiter=10000, match_threshold=3.0):
    x1 = add_z_axis(pts1)
    x2 = add_z_axis(pts2)

    data = np.hstack((x1, x2))
    F, ransac_data = Ransac(data, 8, maxiter, match_threshold, d=18)
    F = F.T

    inliers1 = ransac_data[:, 0:2]
    inliers2 = ransac_data[:, 3:5]

    return F, inliers1, inliers2
```

This is our implementation of the RANSAC normalized 8-point algorithm.

```
def Ransac(data, num_pts, max_iteration, threshold, d=200):
    bestfit = None
    bestcost = np.inf
    best_inlier = None
    model = RansacModel()

    for _ in range(max_iteration):
        train, test = train_test_split(num_pts, data)

        maybemodel = model.fit(train)
        test_err = model.get_error(test, maybemodel)
        num_correct_pts = 0
        inlier_test = []
        for i, err in enumerate(test_err):
            if err < threshold:
                num_correct_pts += 1
                inlier_test.append(test[i])
        inlier_test = np.asarray(inlier_test)

        if num_correct_pts > d:
            thiscost = (data.shape[0] - num_pts - num_correct_pts) * threshold
            thiscost += np.sum(test_err[test_err < threshold])
            if thiscost < bestcost:
                bestcost = thiscost
                best_inlier = np.concatenate((train, inlier_test))

    if best_inlier is None:
        raise ValueError("did not meet fit acceptance criteria")
    bestfit = model.fit(best_inlier)

    return bestfit, best_inlier
```

The function splits the data into two parts: one for the normalized 8-point algorithm to estimate the fundamental matrix and the other for RANSAC to validate the resulting fundamental matrix.

```
def train_test_split(num_sample, data):  
    idx = np.arange(len(data))  
    np.random.shuffle(idx)  
  
    train = data[idx[:num_sample]]  
    test = data[idx[num_sample:]]  
  
    return train, test
```

```
class RansacModel(object):  
    Codeium: Refactor | Explain | Generate Docstring  
    @staticmethod  
    def eight_point_algo(x1, x2): ...  
  
    Codeium: Refactor | Explain | Generate Docstring  
    @staticmethod  
    def normalized_eight_point_algo(x1, x2): ...  
  
    Codeium: Refactor | Explain | Generate Docstring  
    @staticmethod  
    def fit(data): ...  
  
    Codeium: Refactor | Explain | Generate Docstring  
    @staticmethod  
    def get_error(data, F): ...
```

```
@staticmethod
def fit(data):
    data = data.T
    x1 = data[:3, :]
    x2 = data[3:, :]
    return RansacModel.normalized_eight_point_algo(x1, x2)
```

It normalizes the input data and de-normalizes the output of the 8-point algorithm.

```

@staticmethod
def normalized_eight_point_algo(x1, x2):
    n = x1.shape[1]
    if x2.shape[1] != n:
        raise ValueError('Number of points do not match.')

    # Normalization
    x1 = x1 / x1[2]
    x1_mean = np.mean(x1, axis=1)
    S1 = np.sqrt(2) / np.std(x1[:2])
    T1 = np.array([[S1, 0, -S1 * x1_mean[0]],
                   [0, S1, -S1 * x1_mean[1]],
                   [0, 0, 1]])
    x1 = T1 @ x1

    x2 = x2 / x2[2]
    x2_mean = np.mean(x2, axis=1)
    S2 = np.sqrt(2) / np.std(x2[:2])
    T2 = np.array([[S2, 0, -S2 * x2_mean[0]],
                   [0, S2, -S2 * x2_mean[1]],
                   [0, 0, 1]])
    x2 = T2 @ x2

    F = RansacModel.eight_point_algo(x1, x2)

    # De-normalize
    F = T1.T @ F @ T2

    return F / F[2, 2]

```

This is our implementation of the 8-point algorithm.

1. Construct the matrix A .
2. Solve $Af = 0$ by SVD.
3. Enforce the constraint $|F| = 0$ by refining F SVD with another SVD step.


```

@staticmethod
def eight_point_algo(x1, x2):
    n = x1.shape[1]
    if x2.shape[1] != n:
        raise ValueError('Number of points do not match.')

    # 8-point Algorithm
    A = np.zeros((n, 9))
    for i in range(n):
        A[i] = [x1[p, i] * x2[q, i] for p, q in product(range(3), repeat=2)]

    # Solve A*f = 0 to get F
    U, S, V = np.linalg.svd(A)
    F = V[-1].reshape(3, 3)

    # Solve det(F)=0 constraint
    U, S, V = np.linalg.svd(F)
    S[2] = 0
    F = U @ (np.diag(S) @ V)

    return F / F[2, 2]

```

This function computes the Sampson error.

```

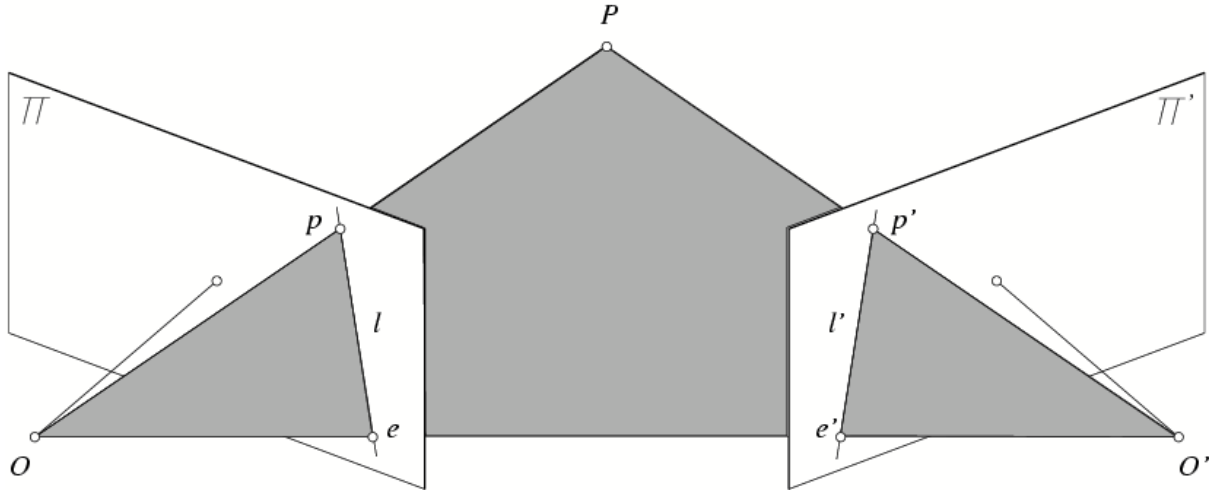
@staticmethod
def get_error(data, F):
    data = data.T
    x1 = data[:3]
    x2 = data[3:]

    Fx1 = F @ x1
    Fx2 = F @ x2
    denom = Fx1[0]**2 + Fx1[1]**2 + Fx2[0]**2 + Fx2[1]**2
    err = (np.diag(x1.T @ F @ x2))**2 / denom

    return err

```

- draw the interest points on you found in step.1 in one image and the corresponding epipolar lines in another



We compute the epipolar lines with the formula $l = Fp'$ and $l' = F^T p$.

We not only draw the interest points and the corresponding epipolar lines computed by our implementation but also draw the epipolar lines generated using OpenCV's built-in function for comparison.

```
def draw_epipolar_lines(gray1, gray2, F, inliers1, inliers2, output_dir, name):
    lines1_cv = cv2.computeCorrespondEpilines(inliers2.reshape(-1, 1, 2), 2, F).reshape(-1, 3)
    lines2_cv = cv2.computeCorrespondEpilines(inliers1.reshape(-1, 1, 2), 1, F).reshape(-1, 3)

    lines2_our = (F @ add_z_axis(inliers1).T).T
    lines2_our = lines2_our / np.linalg.norm(lines2_our, axis=1, keepdims=True)
    lines1_our = (F.T @ add_z_axis(inliers2).T).T
    lines1_our = lines1_our / np.linalg.norm(lines1_our, axis=1, keepdims=True)

    for (lines1, lines2), source in zip(((lines1_cv, lines2_cv), (lines1_our, lines2_our)), ("opencv", "our")):
        plt.figure(figsize=(10, 8))
        image3, image4 = draw_lines(gray2, gray1, lines2, inliers2.astype(np.int32), inliers1.astype(np.int32))
        image5, image6 = draw_lines(gray1, gray2, lines1, inliers1.astype(np.int32), inliers2.astype(np.int32))
        plt.subplot(221), plt.axis('off'), plt.title('First Image (pts2 line)'), plt.imshow(image5)
        plt.subplot(222), plt.axis('off'), plt.title('Second Image (pts2)'), plt.imshow(image6)
        plt.subplot(223), plt.axis('off'), plt.title('First Image (pts1)'), plt.imshow(image4)
        plt.subplot(224), plt.axis('off'), plt.title('Second Image (pts1 line)'), plt.imshow(image3)
        plt.savefig(f"{output_dir}/{name}/{name}_epipolar_{source}")
        plt.close()
```

```
def draw_lines(gray1, gray2, lines, pts1, pts2):
    r, c = gray1.shape
    image1 = cv2.cvtColor(gray1, cv2.COLOR_GRAY2RGB)
    image2 = cv2.cvtColor(gray2, cv2.COLOR_GRAY2RGB)
    for r, pt1, pt2 in zip(lines, pts1, pts2):
        color = tuple(np.random.randint(0, 255, 3).tolist())
        x0, y0 = map(int, [0, -r[2] / r[1]])
        x1, y1 = map(int, [c, -(r[2] + r[0] * c) / r[1]])
        image1 = cv2.line(image1, (x0, y0), (x1, y1), color, 1)
        image1 = cv2.circle(image1, tuple(pt1), 5, color, -1)
        image2 = cv2.circle(image2, tuple(pt2), 5, color, -1)
    return image1, image2
```

- **get 4 possible solutions of essential matrix from fundamental matrix**

Since the essential matrix is composed of R (rotation) and t (translation), we can derive four possible solutions by combining the potential orientations and positions of the camera.

```
E = K1.T @ F @ K2
R1, R2, t = extrinsics(E)
```

```
def extrinsics(E):
    U, S, Vh = np.linalg.svd(E)
    W = np.array([[0, -1, 0], [1, 0, 0], [0, 0, 1]])
    if np.linalg.det(U @ Vh):
        Vh = -Vh
    R1 = U @ W.T @ Vh
    R2 = U @ W @ Vh
    t = U[:, 2:]
    return R1, R2, t
```

- **apply triangulation to get 3D points**

For each possible solution of the essential matrix, a corresponding back-projection matrix can be derived. This results in four sets of reconstructed 3D points.

```
def back_projection(K1, K2, R1, R2, t, pts1, pts2):
    points = []

    for R_, t_ in product((R1, R2), (t, -t)):
        pm1 = K1 @ np.eye(4)[0:3, :]
        pm2 = K2 @ np.hstack((R_, t_))
        point3d = np.zeros([4, pts1.shape[0]])
        A = np.zeros([4, 4])

        for i in range(pts1.shape[0]):
            A[0, :] = pts1[i, 0] * pm1[2, :] - pm1[0, :]
            A[1, :] = pts1[i, 1] * pm1[2, :] - pm1[1, :]
            A[2, :] = pts2[i, 0] * pm2[2, :] - pm2[0, :]
            A[3, :] = pts2[i, 1] * pm2[2, :] - pm2[1, :]

            u, s, vh = np.linalg.svd(A)
            point3d[:, i] = vh[3, :] / vh[3, 3]

        points.append(point3d)

    return points
```

- **find out the most appropriate solution of essential matrix**
Among the four sets of 3D points from the previous step, only one will have all the 3D points in front of both cameras. Therefore, we select the set of 3D points with the highest number of points positioned in front of both cameras.

```
def max_front_points(K1, K2, R1, R2, t, inliers1, inliers2, output_dir, name):
    points = back_projection(K1, K2, R1, R2, t, inliers1, inliers2)
    fronts = [R_[2, :] @ (point3d[0:3, :] + R_.T @ t_) for (R_, t_), point3d in zip(product((R1, R2), (t, -t)), points)]
    point3d = points[np.argmax([np.count_nonzero(front > 0) for front in fronts])]

    fig = plt.figure()
    # ax = Axes3D(fig)
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(point3d[0, :], point3d[1, :], point3d[2, :])
    ax.view_init(elev=40)
    plt.savefig(f"{output_dir}/{name}/{name}_3d_points")
    plt.close()

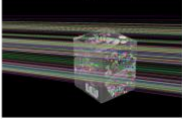
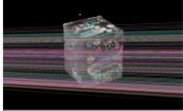
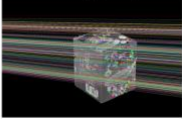
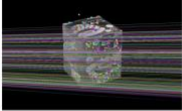
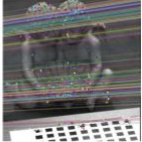
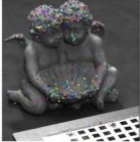
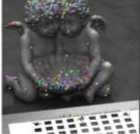
    return point3d
```

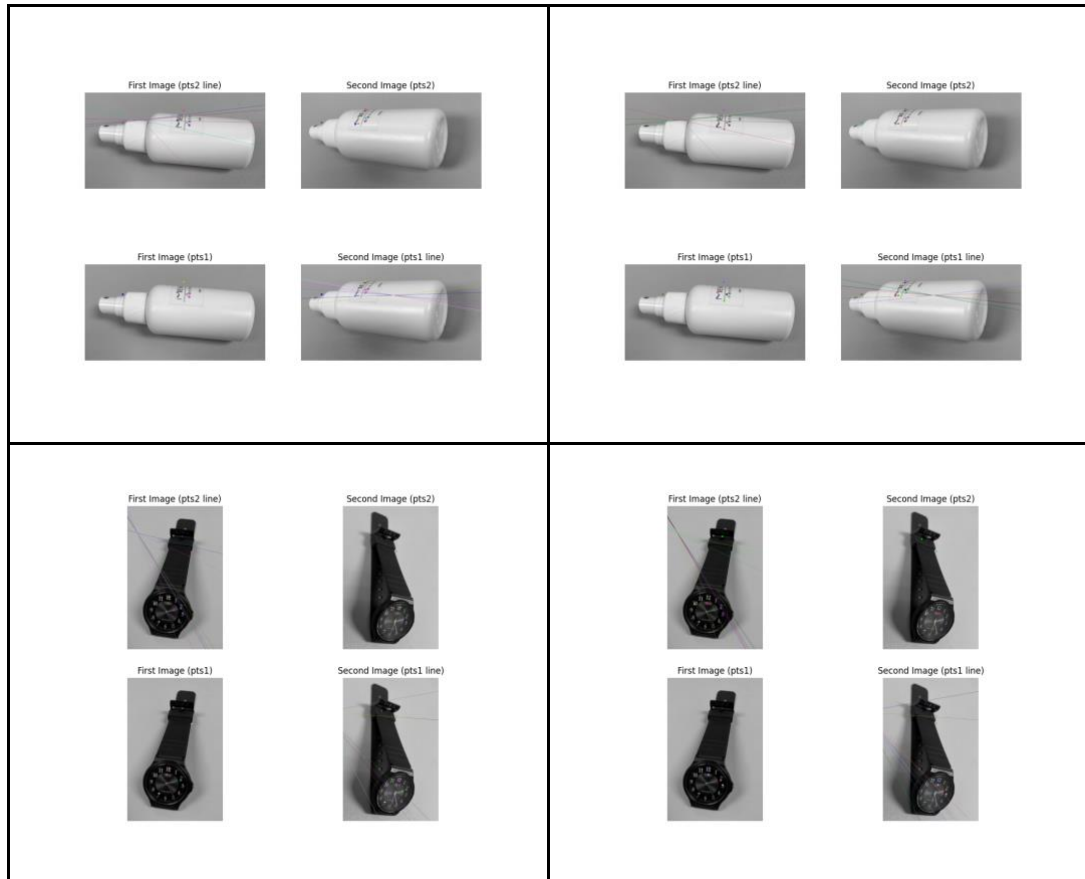
- **use texture mapping to get a 3D model & Use 3d software like Blender to visualize the resulting 3D model(import .obj and .mtl files)**

Please refer to the following part and output.

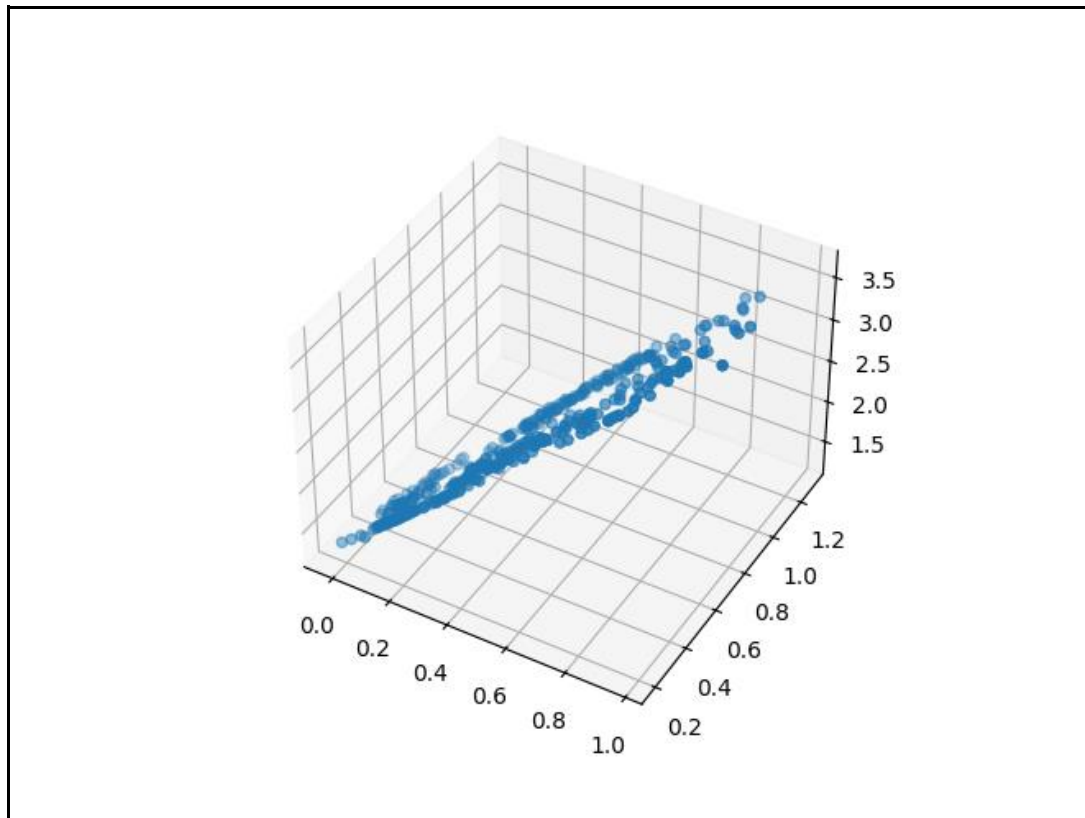
3. Experimental Results

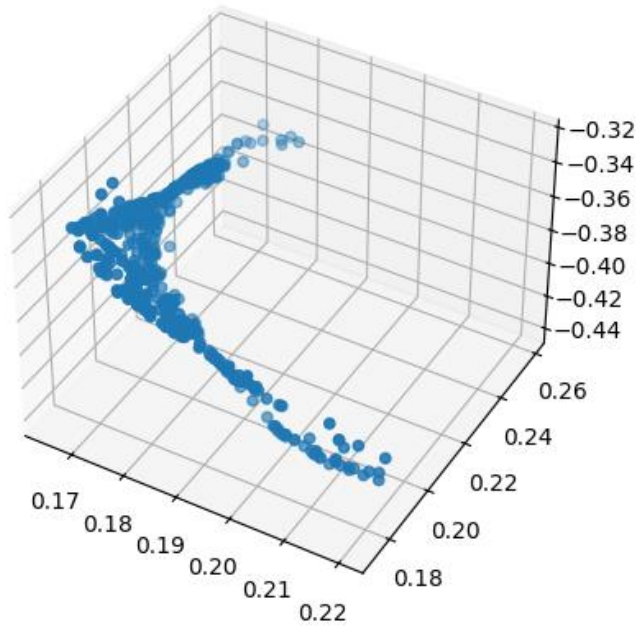
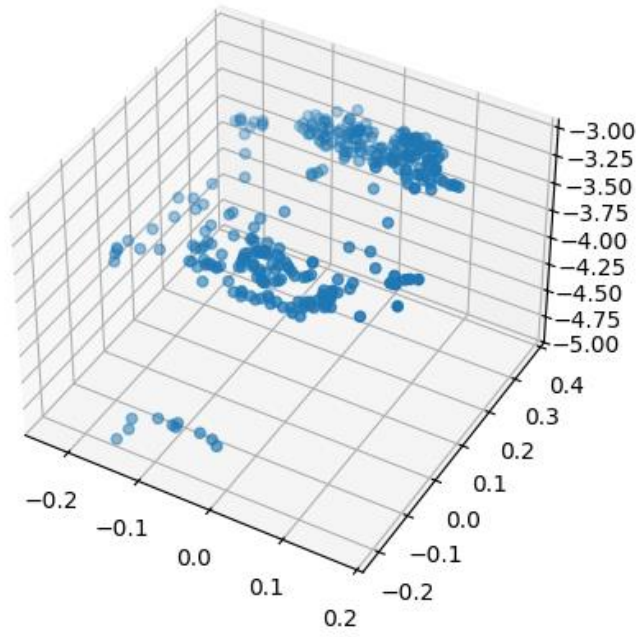
- interest points & epipolar lines

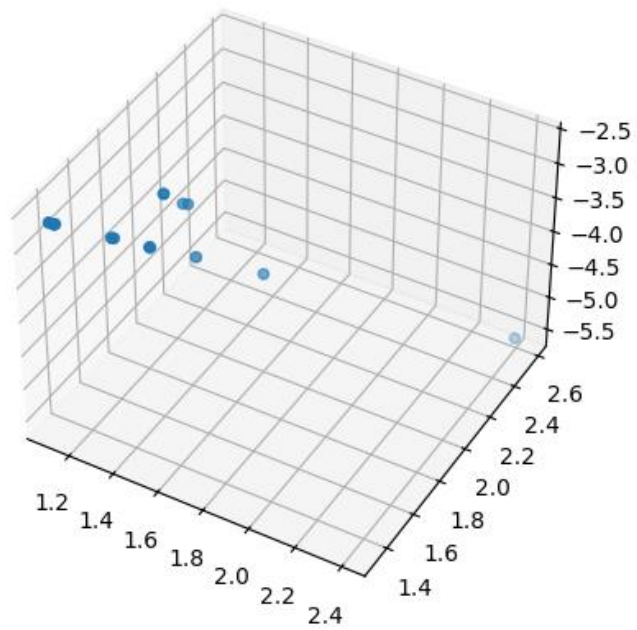
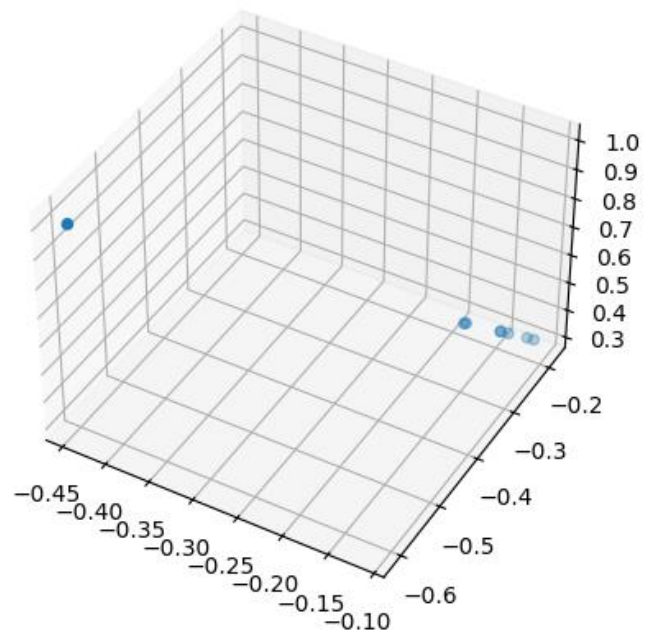
Our Result	OpenCV
First Image (pts2 line)  Second Image (pts2)  First Image (pts1)  Second Image (pts1 line) 	First Image (pts2 line)  Second Image (pts2)  First Image (pts1)  Second Image (pts1 line) 
First Image (pts2 line)  Second Image (pts2)  First Image (pts1)  Second Image (pts1 line) 	First Image (pts2 line)  Second Image (pts2)  First Image (pts1)  Second Image (pts1 line) 
First Image (pts2 line)  Second Image (pts2)  First Image (pts1)  Second Image (pts1 line) 	First Image (pts2 line)  Second Image (pts2)  First Image (pts1)  Second Image (pts1 line) 



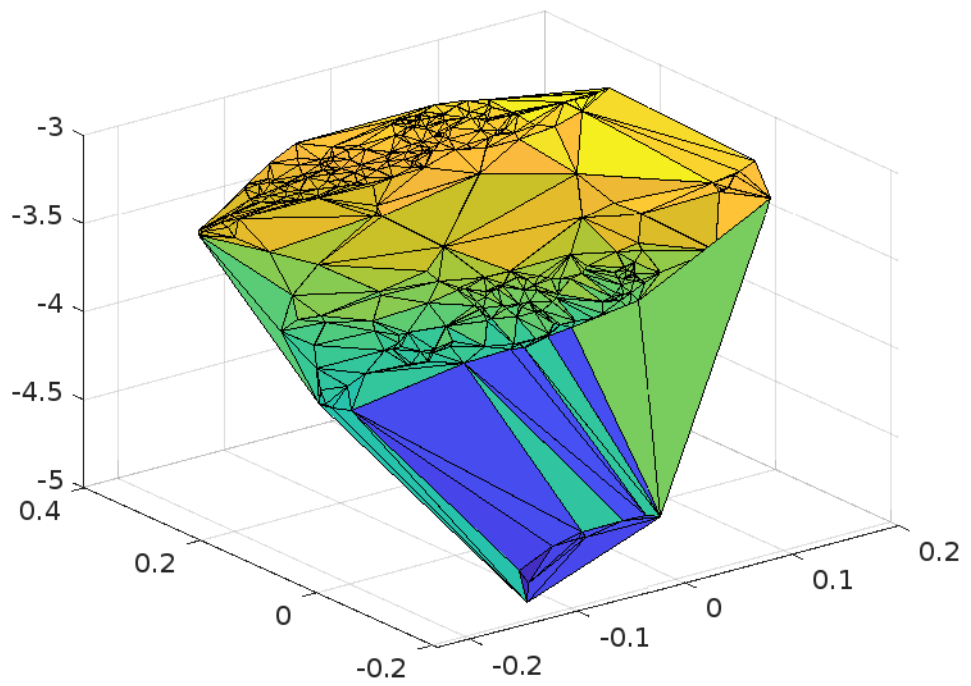
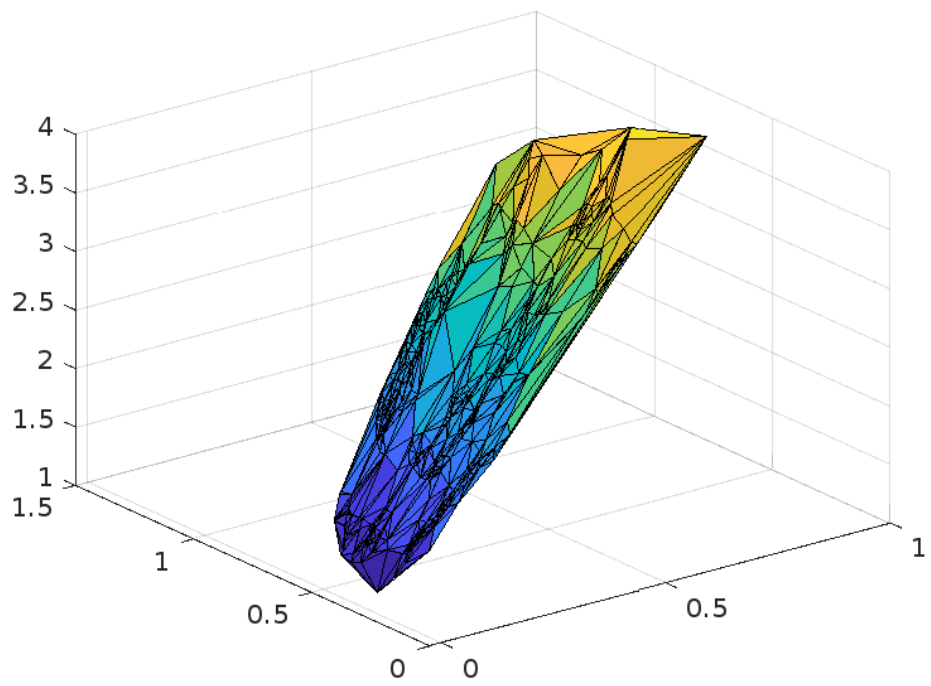
○ 3D points

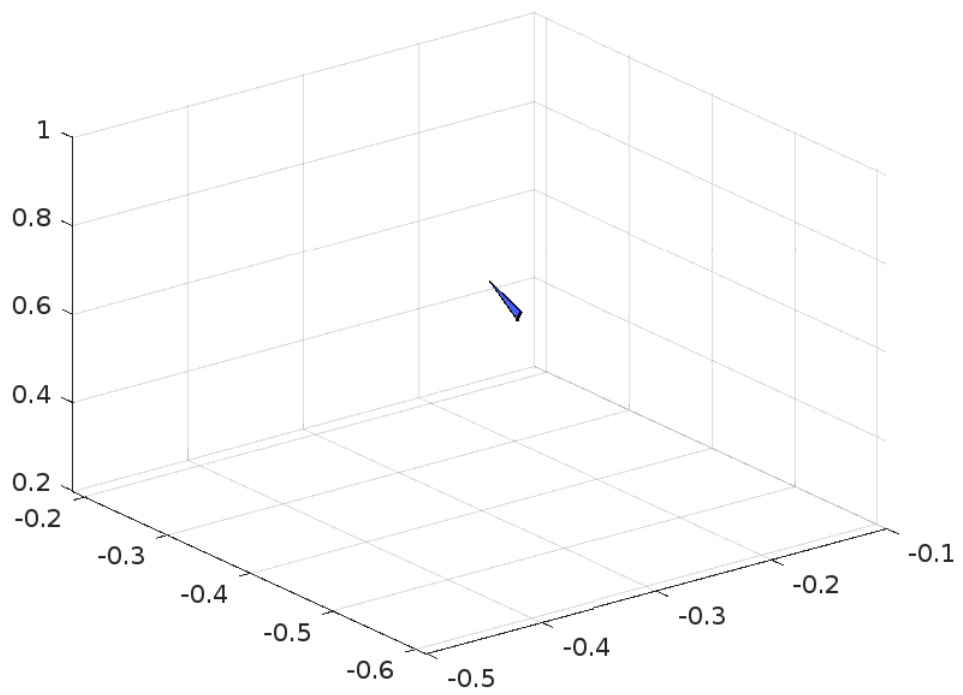
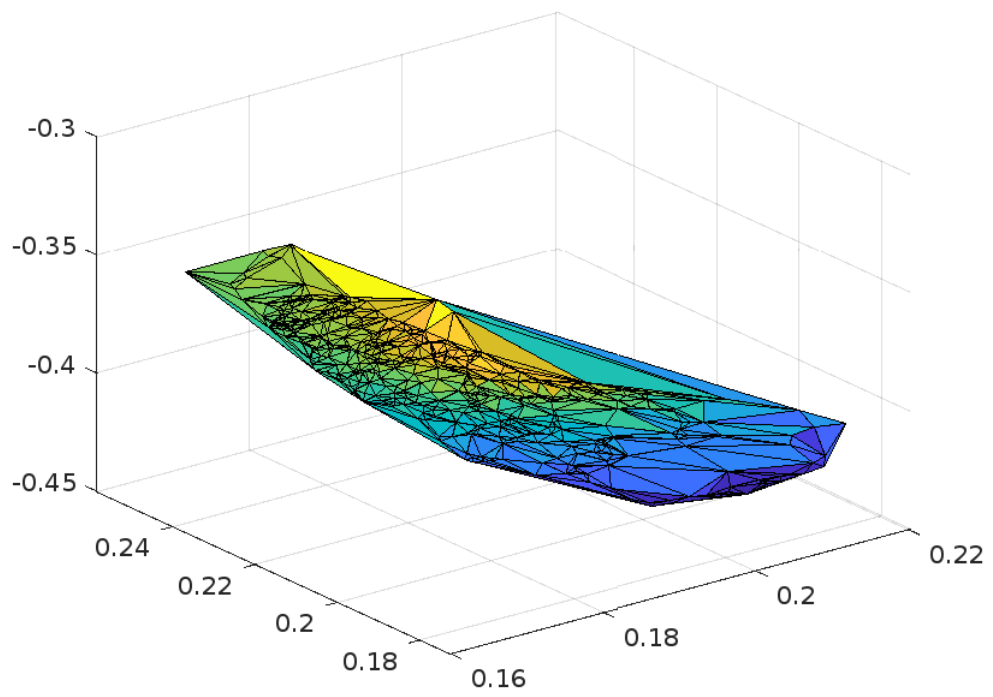


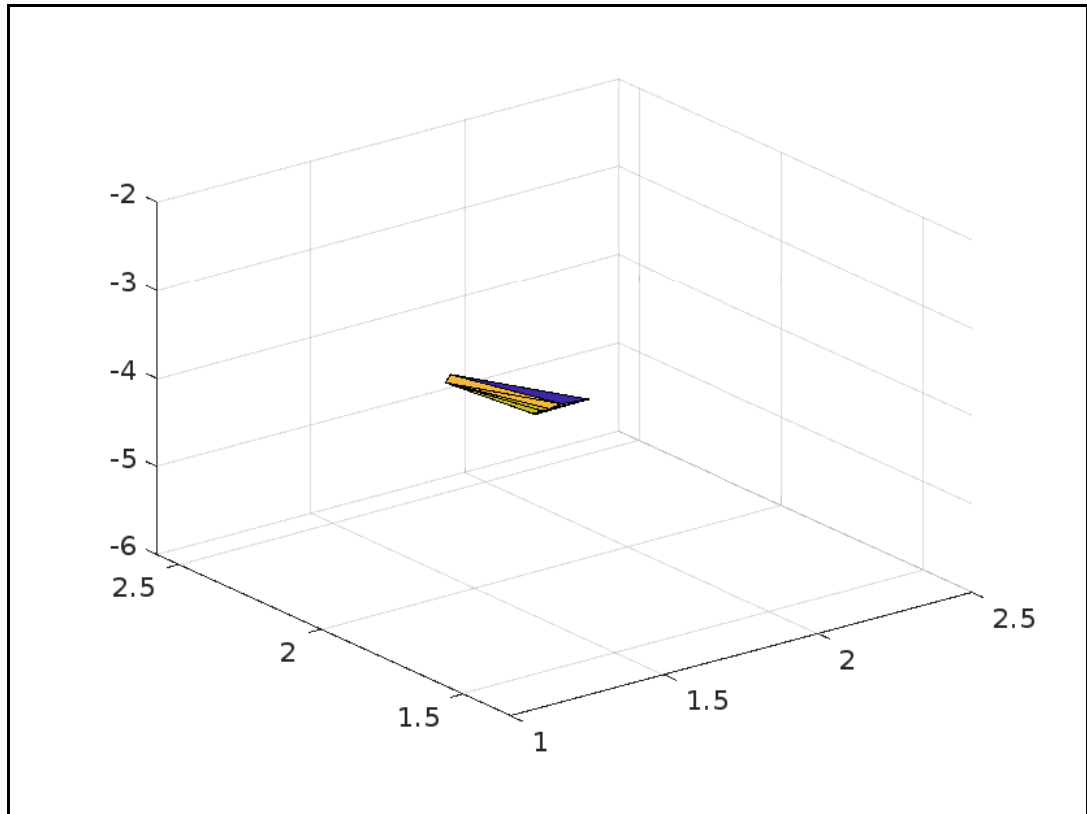




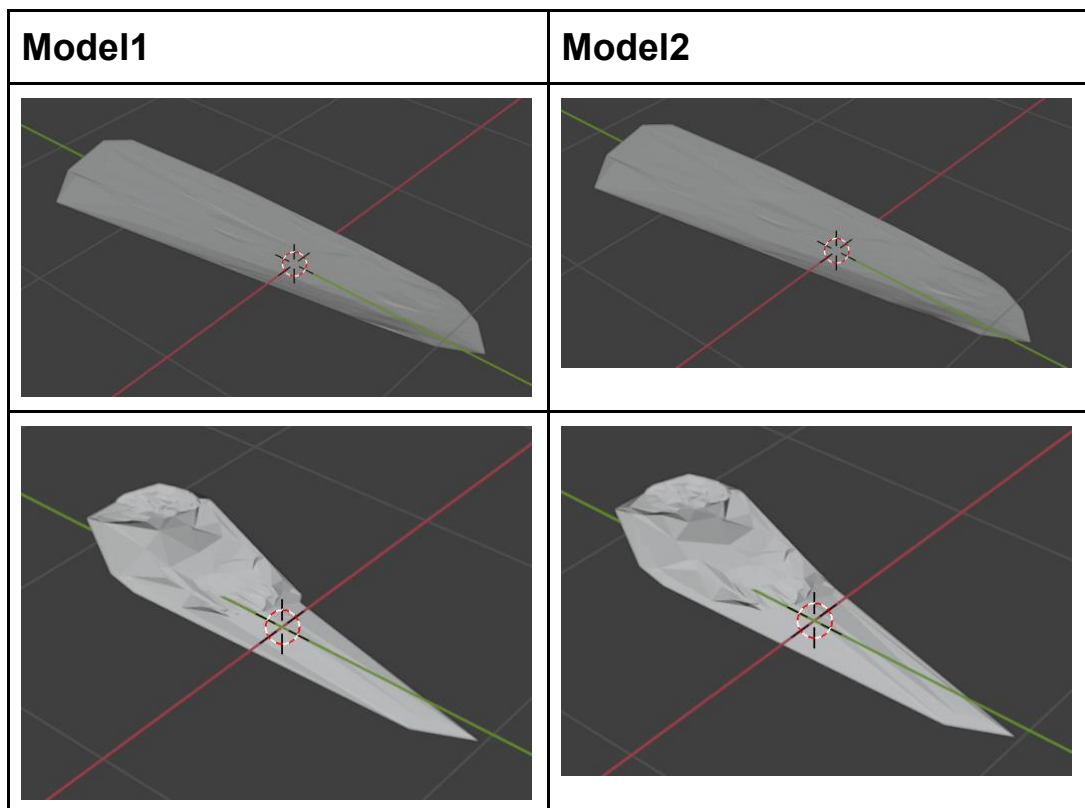
- **mesh triangulation**

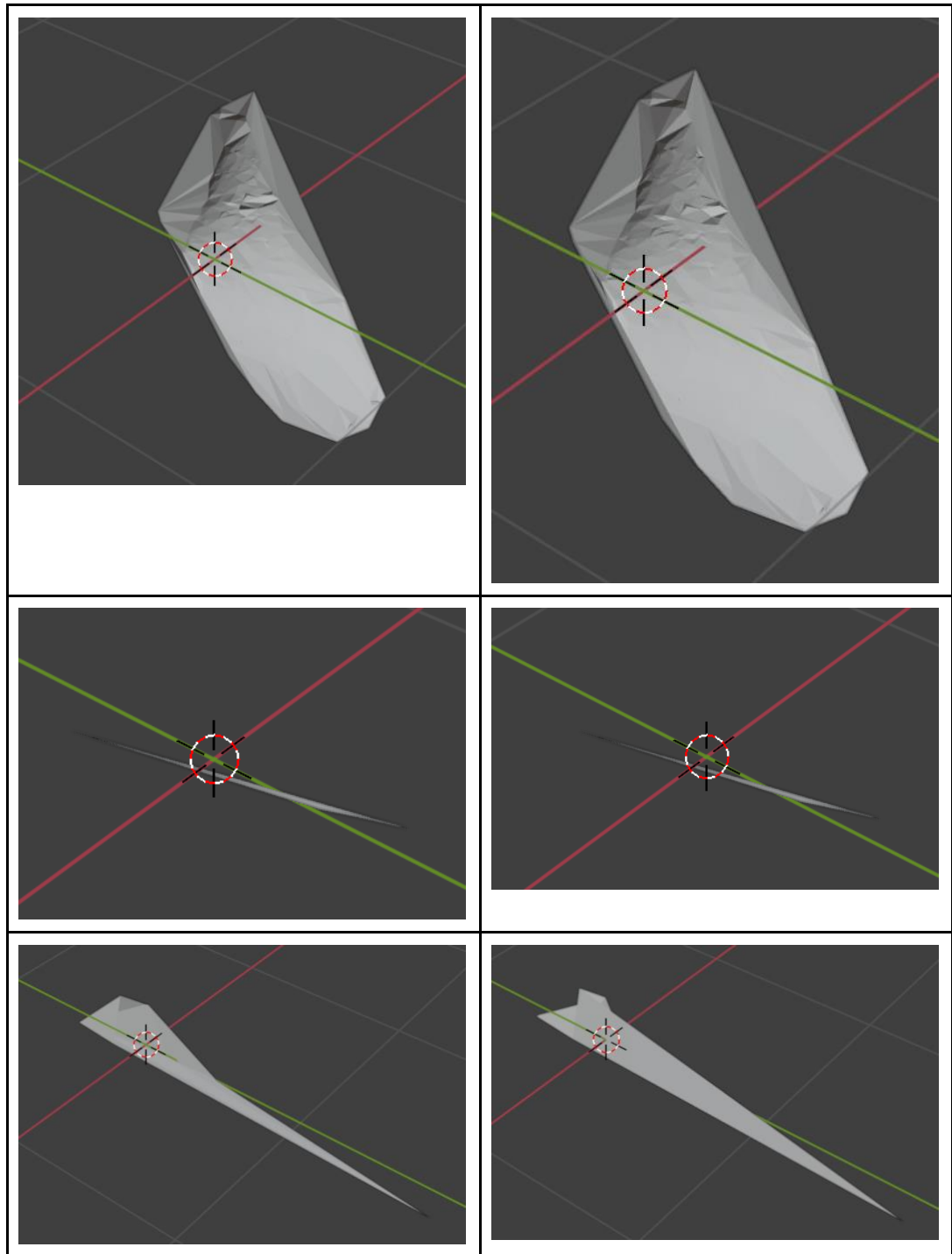






- **3D model in Blender**





4. Discussion

- Although the focus of this homework is not on matching interesting points, it remains a crucial aspect. Without a distance threshold ratio or with one set too large, the matching quality is poor. Conversely, a ratio that is too strict results in too few matches for

- the normalized 8-point algorithm to work. Striking a balance between quality and robustness required careful tuning of the ratio.
- Additionally, the appearance of the object significantly affects robustness. For many common items, it is challenging to extract enough interesting points—examples include ‘Spray.png’ and ‘Watch.png’ from ‘my_data’. After testing various images, we observed that colorful objects and non-monotone backgrounds yield better results.

5. Conclusion

In this homework, we implemented a Structure from Motion (SfM) pipeline to reconstruct 3D structures from 2D image sequences. The process included key steps such as feature matching using SIFT and KNN, estimating the fundamental and essential matrices, deriving 3D points through triangulation, and selecting the most appropriate solution by ensuring the 3D points lie in front of both cameras. By integrating texture mapping and visualizing the 3D model using Blender, we demonstrated the effectiveness of SfM in practical applications.

This exercise highlighted the importance of robust feature matching and careful parameter tuning, as an inappropriate distance ratio negatively impacts the reconstruction. Additionally, object appearance plays a critical role, with colorful and textured images yielding superior results compared to monotone or low-texture ones. Overall, this work underscores the potential of SfM in generating detailed 3D reconstructions from 2D images, while also addressing challenges in real-world scenarios.

6. Work Assignment Plan between Team Members

蔡濟謙	● Do experiments ● Report: Experimental Results, Discussion, and Conclusion
鄭博涵	● Do experiments ● Report: Introduction, Discussion, and Conclusion
陳奎元	● Code implementation

	• Report: Implement Procedure, Experimental Results, Discussion, and Conclusion
--	---

7. Command Line

usage: python3 main.py [OPTION]

- -h, --help show this help message and exit
- --data DATA
- --my_data MY_DATA
- --output OUTPUT

8. Output Explanation

Filename	Description
output/%%/_3D_points.png	Visualization of the 3D points.
output/%%/_epipolar_open cv.png	The epipolar lines drawn with built-in function provided by OpenCV.
output/%%/_epipolar_our.p ng	The epipolar lines drawn with our implementation.
output/%%/_keypoints.png	Visualization of keypoints.
output/%%/_matching.png	Visualization of feature matching.

output/%/%_mesh_triangulation.png	Visualization of mesh
output/%/%[1-2][_*].png	Visualization of the resulting 3D model from different angles
output/%/%_3D_points.csv	The coordinates of the reconstructed 3D points
output/%/%_inliers[1-2].csv	The 2D coordinates of inliers of image [1-2].
output/%/model[1-2].obj	Files for visualization of the resulting 3D model
output/%/model[1-2].mtl	Files for visualization of the resulting 3D model
my_data/chessboard/*	Images for camera calibration